Data Umbrella - Event Board (Open Science Labs)

Candidate Info

- **Name:** Sanvi Shukla
- **GitHub:** https://github.com/sanvishukla
- **Email:** sanvishukla@gmail.com, sanvi.shukla@unb.ca
- **Twitter/X:** Sanvi Shukla
- **University Course:** Computer Science and Engineering (B.Tech)
- **University:** Rajiv Gandhi Institute of Petroleum Technology, Amethi, Uttar Pradesh, India
- **Time Zone:** UTC+5:30 (IST)

Bio:

I am a final-year Computer Science & Engineering student with experience in building applications using Python, Flask, and modern web technologies. Last fall, I was a MITACS Globalink Research Intern at the University of New Brunswick Fredericton, where I developed a Flutter-based application for forest fuel type classification based on a decision-tree model. The system was designed for remote field use, so I implemented an offline-first architecture with a synchronization queue to ensure reliable data upload once connectivity was restored. I also integrated Firebase Authentication and Realtime Database for secure access and data management, and trained ML models using large-scale satellite data to estimate forest attributes such as height, basal area, and growth rate. During my internship at IIT Gandhinagar, I worked on an NLP-based research project (HintQT5) for improving mobile accessibility, where I fine-tuned transformer models and built multiple Flutter applications to test real-time integration.

In addition to my internships, I have built full-stack applications focused on reliability and automation. One of my key projects was the RGIPT Student Hostel Portal, developed using Flask and MySQL, which included secure payment integration using Razorpay with HMAC-based verification and automated PDF generation using ReportLab. I have also been selected for Amazon ML Summer School twice. These experiences have shaped my interest in building systems that handle real-world data, automate workflows, and remain reliable under practical constraints.

Project Overview

- **Project:** Automating Event Data Pipelines and Enhancing Discovery UX for DU Event Board
- **Project Idea/Plan:** The work will be carried out in two parts: improving how event data is handled during contribution, and improving how that data is used on the frontend. On the backend side, the focus is on updating the existing GitHub Actions workflow so that new events are processed as part of pull requests. The workflow will be broken into steps where each event is checked, updated if required, and then written back to the source file. This ensures that once an event is merged, it is already complete and does not need further processing later. Care will be taken to ensure that only new or modified entries are processed so that the workflow remains efficient as the list grows. On the frontend side, the focus is on using this cleaned event list to improve how users explore events. The changes will be built on top of the existing components without breaking current functionality. Features like search and filtering will be integrated in a way that keeps performance stable even as more events are added. The implementation will be done in small steps so that each change can be

tested and reviewed independently. This also ensures that the system remains stable throughout development and that new functionality can be added without affecting existing features.

- **Expected Time (hours):** 350 hrs

Abstract

The DU Event Board currently uses manual inputs and basic validation to manage event data. As the number of events grows, this can make it harder to maintain consistency, avoid duplicate entries, and optimize API usage. Event discovery is also primarily based on simple filters and does not yet include offline access.

This project improves how event data is processed and maintained. On the backend, it extends the existing GitHub Actions workflow to automatically fetch event metadata (image and description), detect duplicates using title and date similarity, and perform geocoding only once per event by writing coordinates back to the source file. These changes prevent duplicate entries, reduce repeated API calls, and reduce the need for manual data fixes.

On the frontend, the project adds full-text search, map-based filtering with a time slider, relative date display, and offline support. It also introduces a guided event submission flow that creates structured GitHub Issues, making it easier for contributors to add events.

Together, these changes reduce manual review work and make it easier for users to search, filter, and access events.

Mentors

Reshama, Ivan

Technical Details

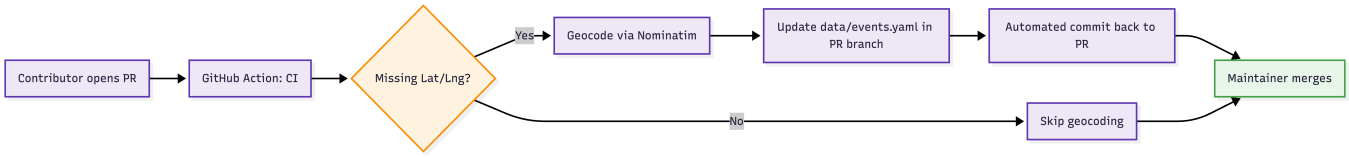
Phase 1: Data Pipeline Reliability & Automation

Before starting the implementation, a step will be added to clean and standardize the data. Fields like format, tags, language, and dates will be converted into a consistent structure. This will include standardizing fields like format (online/in-person/hybrid), normalizing tags and language values, and cleaning URLs and date formats. Basic checks will also be applied to ensure required fields are present and values are valid (for example, start time before end time). The cleaned data will be written back to the source file so that all later steps work on consistent input.

Example of additions in the event schema:

```yaml
format: "Online"
experience_level: "Intermediate"
cost:
  type: "Free"
registration:
  required: true
  deadline: "2026-04-15"
```

1. Reduce API calls in geocoding: Currently, the generate_events_json.py script attempts to geocode events missing "lat/lng" fields during every execution. As the event data grows, this leads to repeated calls to the Nominatim API, risking timeouts and usage policy violations. When an event is added via PR, the CI will geocode it once and then write the coordinates back to the source events.yaml. The Python script will be updated to modify the events.yaml file directly upon successful geocoding. A GitHub Action will then commit these changes back to the branch. This will ensure each event is geocoded exactly once, reducing API dependency to almost zero for existing data.



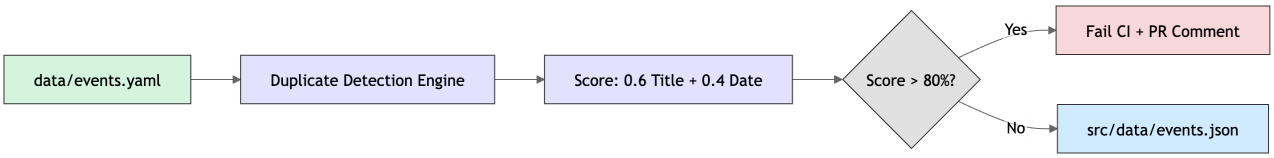
2. Coordinate Validation: The current geocode_location logic will compare the resulting country code from Nominatim with the region field in events.yaml. This prevents "Coordinate Drift". For example, geocoding a city named "London" in Canada when the event is in the UK. Lighthouse CI (@lhci/cli) will be integrated in .github/workflows/ci.yaml to enforce a minimum score of 90 for Performance and Accessibility.

3. Automated Content Enrichment: The current validate_event loop in the Python script will be extended to include a metadata scraper. The aiohttp and asyncio libraries will be used to fetch the image and the description from event URLs during the build process.

   Risk and Mitigation: To avoid CI rate limiting, the current geocode cache architecture will be expanded to a metadata cache to ensure only new/modified events are scraped. Some event hosts (Cloudflare protected) return 403 for automated requests. The enrichment step will be wrapped in a non-fatal try/except. It logs a warning and falls back to the cached value or no image, never blocking the build. A --skip-enrich flag will be available for local development to keep hot reloads instant.

4. Duplicate Detection Engine: A new script will be introduced that will compute "Levenshtein" distances between existing and incoming event titles/URLs. It will look for an exact URL match and a fuzzy title match. If a similarity score > 80% is detected, the CI pipeline will fail with a specific comment flagging the potential duplicate and help prevent data bloat. The CI step will post a structured PR comment showing both events side by side.

   Risk & Mitigation: Recurring meetups/events can share titles but are distinct events. The comparison will use a composite score example: 0.6 * title_ratio + 0.4 * date_proximity, where "date_proximity" will be 1 if within 7 days and 0 beyond 30 days. This reduces false positives for legitimate recurring events.



Phase 2: Distribution & Automation Outputs

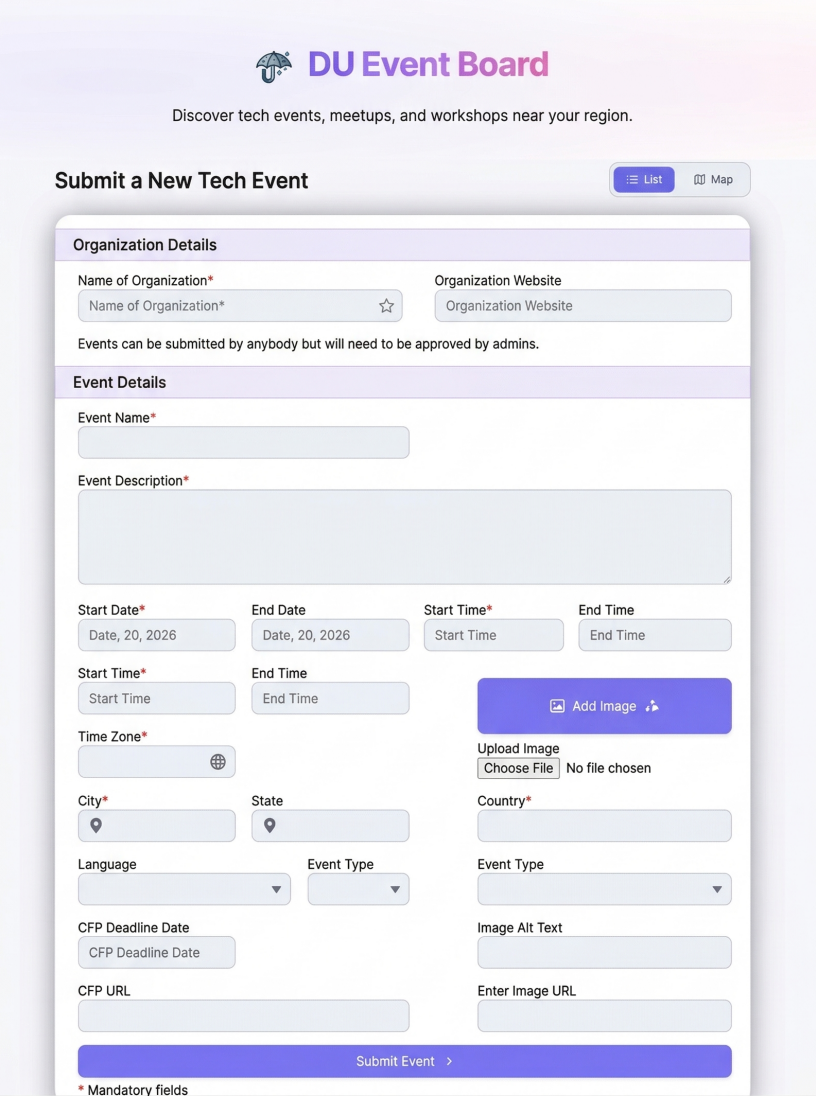
5. Subscription Feature (RSS & iCal): It will create RSS (feed.xml) and iCal (events.ics) files that will let users "subscribe" to the event board as per their filters. Instead of visiting the website every day, they can add the board to their Google Calendar or RSS reader. New events will appear in the user's

calendar or RSS reader. The scripts/generate_events_json.py will be extended to output feed.xml (RSS/Atom) and events.ics (iCalendar) to the public/ directory during each build. The iCal generator will enforce RFC 5545 standards, automatically applying a 2-hour default duration to events missing an end time to ensure proper rendering in Google and Apple Calendars.

6. Automated OG Image Generation: This is a method that Vercel uses to generate preview cards for its blogs. The @vercel/og library was built for build-time image generation. A Node.js script will be added that will render a React component to PNG for every event at build time using @vercel/og (if the image doesn't exist). A hash will be stored in data/.og_cache.json. If that hash matches the previous build, image generation will be skipped, keeping the build times flat as the event count grows.

Phase-3 Contributor Experience (Input Side):

7. Add UI Event Submission: A "Submit Event" form will be created that will open a pre-filled GitHub Issue. This will ensure the events will be reviewed before they can go live.



8. One-Click URL Import: A "Quick Add" field will be added where users paste a Meetup or Eventbrite URL. A serverless proxy via GitHub Actions will scrape Open Graph metadata to pre-populate the contribution form, and the maintainers can merge those events later on.

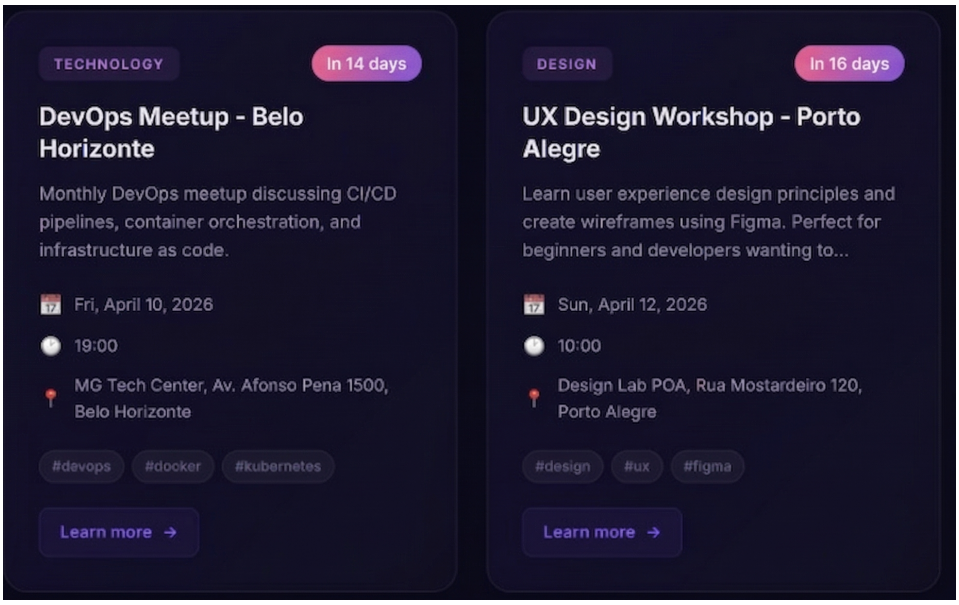
Phase 4: Event Discovery & UX Improvements

9. Build-Time Full-Text Search with Pagefind: Pagefind can rank results by text match instead of file order. The existing SearchBar.jsx will be augmented with a Pagefind-powered path for full-text queries while keeping the in-memory filters.

10. Time-Slider Map Visualisation: A new component positioned as a fixed overlay on the map will be introduced. This will filter mapEvents based on a range slider, showing the number of events as they move from the current week to 6 months out.

11. "in X days" feature: Currently, EventCard.jsx uses a standard toLocaleDateString. Another function will be added to the file. The native Intl.RelativeTimeFormat API will be used to compute "in 3 days" or "yesterday". For events beyond a 30-day threshold, the UI will automatically fall back to the absolute date.

    Risk & Mitigation: Edge cases with timezone offsets between the user's browser and the event's fixed UTC date. All internal calculations will be standardised to the event's local time string before computing offsets, ensuring "Today" is accurate globally.



Phase 5: Offline & App Experience

12. Offline Working: The vite-plugin-pwa will be integrated within vite.config.js. This will allow full offline browsing as well. By caching events.json and the core application shell, users will be able to access the entire website in low-connectivity areas.

Risk & Mitigation: Large events.json files exceeding service worker cache limits on budget devices. A custom precache manifest will be implemented that will prioritise the logic and will only cache the most recent, for example, 1,000 events if the event list grows a lot.

13. Downloading as a Mobile App (PWA): To make the GitHub Pages site "downloadable" like an app, it will need to be converted into a Progressive Web App (PWA). A manifest.json (which defines the app name and icons) will be defined and a small Service Worker (a script that allows the app to work offline). The easiest way to do this is by installing vite-plugin-pwa. It automatically generates these files during the build. Once deployed, users simply visit the URL in their mobile browser:

    iPhone: Tap "Share" → "Add to Home Screen".

    Android: Tap the three dots → "Install App".

    The website will then appear on their home screen with its own icon and will open in a full-screen "app mode" without the browser address bar.

14. "Star" Sync (IndexedDB): To add the star feature for users, the current localStorage of the user will not be used (which is limited to ~5MB) instead IndexedDB will be used. This will also include a Blob-based exporter to generate .ics or .pdf files directly in the browser. Users will be able to receive optional browser-based reminders for events they have starred. This will be implemented using the Notification API and service workers. Reminders will work when the site is active or installed as a PWA, depending on browser support and user permissions.

    Risk & Mitigation: IndexedDB API complexity and browser compatibility. A lightweight wrapper like idb-keyval will be used to ensure state management within the React lifecycle.

Benefits to the Community

This project makes the event board easier to maintain and more useful to use. By automating tasks like geocoding, duplicate detection, and metadata fetching, it reduces manual work for maintainers and avoids repeated errors in the data. Contributors will find it easier to add events through a guided submission flow and URL-based input, which can help bring in more contributions. For users, improved search, map-based filtering, and time-based views make it simpler to find relevant events, while RSS and calendar feeds allow them to follow events without visiting the site regularly. Offline support also ensures the site remains usable even with limited internet access.

Deliverables and Timeline

1. Automated one-time geocoding system with coordinates written back to events.yaml
2. Coordinate validation mechanism to prevent incorrect location mapping
3. Metadata enrichment pipeline (image + description) with caching and fallback handling
4. Duplicate detection system integrated into CI with PR feedback
5. Generation of RSS and iCal subscription feeds
6. Automated OG image generation for event cards
7. UI-based event submission flow using structured GitHub Issues
8. One-click URL import feature for pre-filling event data
9. Integration of full-text search (Pagefind) for improved discovery
10. Map-based visualisation with time slider for event exploration

11. Relative date display (e.g., “in 3 days”)
12. Offline support via PWA (installable app experience)
13. Bookmark/star feature using IndexedDB
14. Updated documentation and setup guides
15. Series of technical blog posts covering major features and implementation decisions

Timeline

Dates	Deliverables/Tasks
May 1 - May 24 (Community bonding period)	-Get better understanding of existing codebase, CI workflows, and data pipeline -Discuss final architecture and priorities with mentors -Improve the existing event schema for implementation - Blog Post 1: Understanding DU Event Board architecture & proposed plan
Week 1 (May 25 - May 31)	- Implement one-time geocoding with write-back to events.yaml - Add GitHub Action to commit coordinates back to PR branch - Initial testing on sample events - Blog Post 2 on optimizing geocoding & reducing API calls
Week 2 (1 June - 7 June)	- Add coordinate validation - Handle edge cases - Write unit tests for geocoding and validation
Week 3 (8 June - 14 June)	- Implement metadata enrichment (image and description scraping) - Add caching layer to avoid repeated requests - Handle failures gracefully (non-blocking CI) - Blog Post 3 on automating event metadata enrichment
Week 4 (15 June - 21 June)	- Build duplicate detection flow - Integrate into CI with PR comments for flagged duplicates - Tune the thresholds to reduce false positives - Add test cases
Week 5 (22 June - 28 June)	- Implement RSS + iCal feed generation -Validate feeds with external tools (Google Calendar, RSS readers) - Blog Post on making events subscribable (RSS & iCal)
Week 6 (29 June - 5 July)	- Add automated OG image generation - End-to-end testing of the full pipeline - Buffer time for bug fixes
Week 7 (6 July - 12 July) Mid-term evaluation	- Documentation for all features introduced - Mid-term report submission and blog post 5 on the same
Week 8 (13 July - 19 July)	- Implement UI Event Submission -Add One-click URL import

Dates	Deliverables/Tasks
Week 9 (20 July - 26 July)	-Integrate the above with submission flow - Improve validation before submission - Blog Post 6 on Reducing friction in event contributions
Week 10 (27 July - 2 August)	- Integrate Pagefind for full-text search - Improve search relevance & filtering - Optimize performance
Week 11 (3 August - 9 August)	- Implement map time-slider visualisation - Add relative date display (“in X days”) - Add PWA support (offline mode + installable app)
Week 12 (10 August - 16 August)	- Implement IndexedDB-based star/bookmark feature - Final testing, bug fixes, and performance tuning - Complete documentation and usage guides - Final Blog Post: Complete project summary, demos, and future work

Previous Contributions to the Project

I have contributed to the DU Event Board on both the frontend and the data pipeline. My work includes adding the map view with geolocation and clustering, improving the geocoding logic to avoid repeated API calls, implementing shareable filter URLs, and building filtering and date-range features. Through this, I have understood how event data moves from submission to display and where issues still exist, such as inconsistent metadata and repeated processing steps.

Number of PRs Merged: 8

Open PR: 4

Closed (with future scope): 1

Issues resolved: 3

Pull Requests

Pull Request Title/Number	Status
[FEATURE]Add Interactive Map View & Zero-Friction Geocoding (#43)	Merged
Feat: Implement Shareable Filter URLs (Deep Linking) (#48)	Merged
feat: Add about section at the end of page (#52)	Merged
feat: Implement start/end date filter (#60)	Merged
feat: Improve the event geocoding pipeline to reduce API calls (#87)	Merged
feat: improve map view with automatic geolocation and fit-bounds logic (#89)	Merged
fix: prevent start date from being after end date in custom range (#112)	Merged

Pull Request Title/Number	Status
feat: add horizontal List view parallel to Grid view (#113)	Merged
feat: Add interactive event generator CLI (#51)	Closed
Feat: Added near-me and clustering feature to Map view (#63)	Open
Feat/Created About Us page and Sponsors page (#88)	Open
feat: implement hybrid search engine (precision + typo-tolerance) (#91)	Open
Feat/automate dead-link validation via GitHub Actions (#106)	Open

Why This Project?

This project fits well with the kind of work I enjoy, that is, building systems that handle real data and make things easier for both users and contributors. In my previous projects, I've worked on processing structured data, integrating APIs, and building applications that are actually used in practice, so this feels like a natural extension of that experience.

What I like about this project is that it's not limited to just frontend or backend work. It involves improving how data flows through the entire system, from how events are added and validated to how they are finally displayed and explored. Since I've already contributed features like the map view, filtering, and deep linking, I've had the chance to understand how the platform works in reality, not just in theory. Because of this, I'm familiar with some of the current limitations, like inconsistent metadata, areas where contributor workflows can be smoother, and parts of the GitHub Actions pipeline that can be made more efficient. This puts me in a position where I can start making meaningful improvements without spending too much time getting up to speed.

I'm particularly interested in improving the automation side of the project, things like automatically enriching event data, detecting duplicates, and making geocoding more efficient. These are small changes individually, but together they can reduce a lot of manual effort and make the platform more reliable and easier to maintain.

Availability

I will complete my degree this summer and have no other professional commitments during the GSoC period. Therefore, I can dedicate 35-40 hours per week to the project. I will keep some time of the week before the mid-term report and final report as a buffer period, where I will make sure to catch up with any incomplete work.

Post-GSoC

After GSoC, I'd like to continue contributing to the project and build on the work done during the program.

I plan to keep improving the automation workflows so they remain reliable as the platform grows. I'm also interested in taking the "no-code contribution" idea further by turning it into a more guided and user-friendly submission flow, so that even non-technical users can easily add events. Another future scope is making the website more accessible (by adding tags for screenreaders, improving color contrast, etc.). Alongside development, I plan to continue writing blog posts to document new features, explain design

decisions, and share progress. This will help future contributors understand the system more easily and lower the barrier to entry.

I'd also be happy to stay involved with the community by reviewing PRs and helping new contributors get started, since I know how valuable that support can be.